ALEXANDRIA NATIONAL UNIVERSITY | COLLEGE OF COMPUTING

CYBER SECURITY PROGRAM | 00304N: DISTRIBUTED PROCESSING

Lab 1: Building a Reflection-Based RPC System

Implementation and Design Report

Go (standard library only) • TCP • JSON • Reflection

Course	00304N: Distributed Processing
Instructor	Dr. Laurine Armeya
Teaching Assistants	Staff Members
Module	`lab1` (Go 1.20+, verified with Go 1.23.4)
Date	October 20, 2026
Status	**All tasks implemented; all tests pass**

Team

#	Name	Student ID
1		
2		
3		

> **Scope of this report.** It explains what was built, how each assignment task works, why the design choices were made, how the system is used, and how it was verified. Every function in `rpc/rpc.go` carries an explanatory Go doc comment with its purpose and a usage example.

1. Executive Summary

This report documents a complete implementation of the Lab 1 Remote Procedure Call (RPC) system. The system lets a client invoke exported methods on a server object over a TCP connection using JSON messages, with the server locating and calling the method through Go's `reflect` package at runtime.

The assignment template already contained the wire types, the reflection bridge skeleton, the client constructor, and the test suite. The required work was concentrated in seven functions: `Server.Start`, `Server.Address`, `Server.Stop`, `validateRequest`, `Server.handle`, `Server.invoke`, and `Client.Call`. All seven are implemented without changing any existing function signature or JSON field name, and all original TODO comments are preserved.

Results at a glance

- **13/13** top-level tests pass (23 test cases including subtests).
- The test suite also passes with the Go **race detector**, confirming that the lifecycle and connection bookkeeping are concurrency-safe.
- The runnable demo (`go run ./cmd/ping`) prints `Pong1` through `Pong4`.
- Only the Go standard library is used; there are no external dependencies.

Deliverables

Deliverable	Location	Purpose
RPC library	`lab1/rpc/rpc.go`	Server, client, protocol types, validation, reflection invocation, lifecycle.
Test suite	`lab1/rpc/rpc_test.go`	Provided specification; unchanged and fully passing.
Demo program	`lab1/cmd/ping/main.go`	End-to-end usage example: starts a server, calls Ping four times.
Module file	`lab1/go.mod`	Module `lab1`, Go 1.20, no dependencies.
This report	`Lab1_RPC_Report.pdf`	Design, implementation walkthrough, usage, and verification evidence.

2. Project Layout

```
lab1/
|-- go.mod                  module lab1, Go 1.20, standard library only
|-- cmd/
|   `-- ping/
|       `-- main.go         runnable client/server demo
`-- rpc/
    |-- rpc.go              RPC implementation (server, client, protocol, reflection)
    `-- rpc_test.go         provided test suite (13 top-level tests)
```

The layout follows the conventional Go split between a reusable package (`rpc`) and an executable (`cmd/ping`). The library has no knowledge of `main`; the demo depends on the library, which keeps the package testable and reusable.

3. System Architecture

3.1 Components

Component	Responsibility
`Server`	Owns the receiver object, the TCP listener, and lifecycle state. Exposes exported methods.
`Server.Start`	Creates the listener, records lifecycle state, starts the accept loop in the background.
`Server.acceptLoop`	Accepts connections forever; each connection becomes a tracked worker goroutine.
`Server.handle`	Reads exactly one JSON request, validates it, invokes it, writes one JSON response, closes.
`validateRequest`	Enforces method-name, argument-count, and size limits before reflection or user code.
`Server.invoke`	Finds the method with reflection, decodes each argument into its Go type, calls it, and normalises results.
`callMethodSafely`	Converts a panic in user code into a normal error.
`Client.Call`	Opens one connection per call, applies a deadline, sends one request, returns the result or remote error.
`request` / `response`	The JSON wire types. `json.RawMessage` defers argument decoding until reflection knows the target type.

3.2 Wire Protocol

Each call is one request and one response on one TCP connection. The server closes the connection after writing the response. The JSON field names are fixed by the assignment.

Request	Successful response	Error response

`{`
`  "method": "Add",`
`  "args": [2, 3]`
`}`
	`{`
	`  "ok": true,`
	`  "value": 5`
	`}`
		`{`
		`  "ok": false,`
		`  "error": "unknown method \"Missing\""`
		`}`

Protocol invariants. The client sends exactly one request per connection and the server sends exactly one response before closing. There is no length header: the JSON decoder reads one complete, self-delimiting value. Every response carries `ok`; exactly one of `value` or `error` is meaningful.

> **Why JSON?** It is language-neutral, human-readable, and trivially debuggable with tools such as `nc` or a browser. The cost is a larger payload than a binary encoding, which is an acceptable trade-off for a teaching system where clarity matters more than throughput.

3.3 End-to-End Call Sequence

```
Client.Call("Add", &sum, 20, 22)
   │
   ├─ 1. net.DialTimeout(Address, Timeout)        one fresh TCP connection
   ├─ 2. conn.SetDeadline(now + Timeout)          bounds connect + write + read
   ├─ 3. encode {"method":"Add","args":[20,22]}
   ▼
Server.acceptLoop (background goroutine, one per running server)
   ├─ 4. listener.Accept()                        blocks until a client connects
   ├─ 5. workers.Add(1)                           register the in-flight call
   └─ 6. go handle(conn)                          one goroutine per connection
   ▼
Server.handle(conn)
   ├─ 7. LimitReader + json.Decode                exactly one request, size-capped
   ├─ 8. validateRequest(req)                     reject abuse before reflection
   ├─ 9. invoke(req)
   │       ├─ reflect.MethodByName("Add")         exported methods only
   │       ├─ check NumIn() == len(args)
   │       ├─ json.Unmarshal each raw arg         into the declared Go type
   │       ├─ callMethodSafely(...)               panic becomes an error
   │       └─ normalise return values             value / error / value+error / none
   ├─ 10. encode {"ok":true,"value":42}
   └─ 11. defer conn.Close()                      protocol: one call per connection
   ▼
Client.Call
   ├─ 12. decode response
   ├─ 13. ok == false  → return errors.New(server message)
   └─ 14. ok == true   → json.Unmarshal(value, out); nil out is accepted
```

4. Implementation Walkthrough

Each subsection states the assignment requirement, shows the essential code, and explains why the implementation satisfies it. Full doc comments, including usage examples, are in `rpc/rpc.go`.

4.1 Task 1 — Start the RPC Server

Requirement: create a TCP listener, return startup errors, reject a second `Start`, save the listener and lifecycle state, launch the accept loop in a goroutine, and return without waiting for a client. Port `0` must let the OS choose a port.

```
func (s *Server) Start(address string) error {
    s.mu.Lock()
    defer s.mu.Unlock()

    if s.listener != nil {
        return errors.New("server already started")
    }
    listener, err := net.Listen("tcp", address)
    if err != nil {
        return err
    }

    stopped := make(chan struct{})
    acceptDone := make(chan struct{})
    s.listener = listener
    s.stopped = stopped
    s.acceptDone = acceptDone

    go s.acceptLoop(listener, stopped, acceptDone)
    return nil
}
```

Why it works. The mutex makes the "already running" check and the state update atomic, so two concurrent starts cannot both succeed. Fresh `stopped` and `acceptDone` channels are created on *every* start: channels cannot be closed twice, so reusing them would make a restart panic. The accept loop is passed the listener and channels as arguments, so it never reads mutable fields without the lock. Because the loop runs in a goroutine, `Start` returns immediately. `net.Listen` resolves `:0` to a real port, which is later exposed by `Address`.

4.2 Task 2 — Return the Server Address

Requirement: return the actual listening address, including the OS-selected port; return an empty string before start and after stop; read shared state safely.

```
func (s *Server) Address() string {
    s.mu.Lock()
    defer s.mu.Unlock()

    if s.listener == nil {
        return ""
    }
    return s.listener.Addr().String()
}
```

Why it works. `listener.Addr()` reports the address the operating system actually bound, not the requested string, so `127.0.0.1:0` becomes something like `127.0.0.1:41237`. The mutex prevents a data race with `Start` and `Stop`, which mutate `s.listener`. A nil listener represents both "not started" and "stopped", so both cases correctly return `""`.

4.3 Task 3 — Stop the Server

Requirement: stop accepting, close the listener, signal the accept loop, wait for the loop and for active handlers, leave the server restartable, and be safe before start and when called repeatedly.

```
func (s *Server) Stop() error {
    s.mu.Lock()
    if s.listener == nil {          // never started, already stopped, or stopping
        s.mu.Unlock()
        return nil
    }
    listener := s.listener
    stopped := s.stopped
    acceptDone := s.acceptDone

    s.listener = nil                // Address() now returns ""
    s.stopped = nil
    s.acceptDone = nil
    close(stopped)                  // signal the accept loop
    s.mu.Unlock()

    _ = listener.Close()            // unblocks Accept with an expected error
    <-acceptDone                    // no new workers can be added after this
    s.workers.Wait()                // let in-flight handlers finish
    return nil
}
```

Why it works. Clearing the listener first makes the operation idempotent: a second `Stop` sees nil and returns successfully. Closing `stopped` tells the loop to stop, while closing the listener unblocks a goroutine currently sleeping in `Accept`. The ordering of the two waits is essential: `acceptDone` is awaited before `workers.Wait`, so once the accept loop has exited no new `workers.Add` can race with `Wait`. Waiting for the workers means an in-flight call still finishes and receives its result (`TestStopWaitsForActiveHandler`). Since the state is reset, a later `Start` creates fresh channels and succeeds.

4.4 Task 4 — Validate Requests

Requirement: reject empty or unsafe method names, more than `maxArguments` arguments, a single argument larger than `maxArgumentBytes`, or arguments totaling more than `maxRequestBytes`.

```
func validateRequest(req request) error {
    if req.Method == "" {
        return errors.New("method cannot be empty")
    }
    for _, character := range req.Method {
        if unicode.IsSpace(character) || unicode.IsControl(character) {
            return fmt.Errorf("method %q cannot contain whitespace or control characters",
                req.Method)
        }
    }
    if len(req.Args) > maxArguments {
        return fmt.Errorf("too many arguments: %d (maximum %d)", len(req.Args), maxArguments)
    }

    total := 0
    for index, argument := range req.Args {
        if len(argument) > maxArgumentBytes {
            return fmt.Errorf("argument %d is larger than %d bytes", index, maxArgumentBytes)
        }
        total += len(argument)
    }
    if total > maxRequestBytes {
        return fmt.Errorf("request arguments are larger than %d bytes", maxRequestBytes)
    }
    return nil
}
```

Why it works. Iterating over runes (not bytes) makes the whitespace/control check correct for non-ASCII input. The per-argument and total checks use the *encoded* lengths of `json.RawMessage`, which is exactly what the server would process. Validation runs before `invoke`, so hostile input is rejected before any reflection, allocation of parameter values, or application code runs. This is a small but real denial-of-service defence.

Limit	Value	Protects against
`maxArguments`	8	Calls with an unreasonable parameter count.
`maxArgumentBytes`	1 MiB	One huge argument consuming memory.
`maxRequestBytes`	4 MiB	Many medium arguments consuming memory.
`defaultCallTimeout`	5 s	Clients waiting forever on a stuck server.

4.5 Task 5 — Handle RPC Calls

Requirement: read one JSON request, answer malformed JSON with an error, validate, invoke, return success or error JSON, convert method panics into errors, and close the connection. A bad request must neither crash the server nor leave the client waiting.

```
decoder := json.NewDecoder(bufio.NewReader(io.LimitReader(connection, maxRequestBytes+1)))
var req request
if err := decoder.Decode(&req); err != nil {
    _ = encoder.Encode(response{Error: fmt.Sprintf("invalid request: %v", err)})
    return
}
if err := validateRequest(req); err != nil {
    _ = encoder.Encode(response{Error: err.Error()})
    return
}
result, err := s.invoke(req)
if err != nil {
    _ = encoder.Encode(response{Error: err.Error()})
```

```
    return
}
payload, err := json.Marshal(result)
if err != nil {
    _ = encoder.Encode(response{Error: fmt.Sprintf("encode result: %v", err)})
    return
}
_ = encoder.Encode(response{OK: true, Value: payload})
```

Why it works. The protocol is one request per connection, so a single `Decode` is sufficient. `io.LimitReader` caps the bytes the decoder may consume; adding one byte lets an over-limit body fail during decoding instead of being silently truncated and accepted. Every failure path writes a JSON error response before returning, so the client always receives an answer. A deferred `recover` is kept as a final safety net, although method panics are already handled inside `callMethodSafely`. The deferred `connection.Close()` guarantees the socket is released.

4.6 Task 6 — Invoke Methods with Reflection

Requirement: find the exported method, error clearly when it is missing, check the argument count, decode each JSON argument into the parameter type, call the method, support all required return shapes, surface non-nil errors, and convert panics into errors.

```
method := reflect.ValueOf(s.receiver).MethodByName(req.Method)
if !method.IsValid() {
    return nil, fmt.Errorf("unknown method %q", req.Method)
}
typeOfMethod := method.Type()
if typeOfMethod.NumIn() != len(req.Args) {
    return nil, fmt.Errorf("method %q expects %d arguments, got %d",
        req.Method, typeOfMethod.NumIn(), len(req.Args))
}

arguments := make([]reflect.Value, len(req.Args))
for index, raw := range req.Args {
    argument := reflect.New(typeOfMethod.In(index))   // *T
    if err := json.Unmarshal(raw, argument.Interface()); err != nil {
        return nil, fmt.Errorf("argument %d: %v", index, err)
    }
    arguments[index] = argument.Elem()                // T
}

outputs, err := callMethodSafely(method, arguments)
if err != nil {
    return nil, err
}

errorType := reflect.TypeOf((*error)(nil)).Elem()
if last := outputs[len(outputs)-1]; last.Type().Implements(errorType) {
    if !isNilValue(last) {
        return nil, last.Interface().(error)
    }
    outputs = outputs[:len(outputs)-1]
}
if len(outputs) == 0 {
    return nil, nil
}
return outputs[0].Interface(), nil
```

Why it works. `MethodByName` only returns exported methods, so the server cannot accidentally expose private helpers. For a bound method value, `NumIn()` counts only declared parameters (the receiver is already bound). `reflect.New(T)` gives a `*T` that `json.Unmarshal` can fill; `Elem()` then yields the `T` that `Call` expects. Handling the last return value separately supports all required shapes:

Method shape	Reflection outputs	Response
`NoResult()`	0 values	`ok:true`, no value
`NoArgs() string`	1 value	value
`Add(int,int) int`	1 value	value
`OnlyError(bool) error`	1 error	nil error → no value; non-nil → error
`Ping(int) (string, error)`	value + error	error if non-nil, else value

`callMethodSafely` recovers a panic and returns it as an ordinary error. Without it, a buggy remote method would unwind through the handler and could terminate the process. `isNilValue` checks nilable kinds before calling `IsNil`, which would panic on an integer or string.

4.7 Task 7 — Implement the RPC Client

Requirement: reject an empty method name, open one connection, apply the timeout as a connection and I/O deadline, encode one request, decode one response, return remote errors, decode the value into `out` (accepting nil), close the connection, and produce useful errors.

```
if method == "" {
    return errors.New("method cannot be empty")
}
encodedArgs := make([]json.RawMessage, len(args))
for index, argument := range args {
    raw, err := json.Marshal(argument)
    if err != nil {
        return fmt.Errorf("encode argument %d: %w", index, err)
    }
    encodedArgs[index] = raw
}

server, err := net.DialTimeout("tcp", c.Address, timeout)
if err != nil {
    return fmt.Errorf("connect to %s: %w", c.Address, err)
}
defer server.Close()
if err := server.SetDeadline(time.Now().Add(timeout)); err != nil {
    return fmt.Errorf("set deadline: %w", err)
}

if err := json.NewEncoder(server).Encode(request{Method: method, Args: encodedArgs}); err != nil {
    return fmt.Errorf("encode request: %w", err)
}
var remote response
if err := json.NewDecoder(bufio.NewReader(server)).Decode(&remote); err != nil {
    return fmt.Errorf("decode response: %w", err)
}
if !remote.OK {
    return errors.New(remote.Error)
}
if out == nil {
    return nil
}
if err := json.Unmarshal(remote.Value, out); err != nil {
    return fmt.Errorf("decode response value: %w", err)
}
return nil
```

Why it works. Marshalling each argument separately produces the exact `[]json.RawMessage` the protocol expects; the server needs the raw bytes because only it knows the parameter types. `DialTimeout` bounds connection setup and `SetDeadline` bounds the later write and read, so a slow method cannot block the caller forever (`TestClientTimeout`). A response with `ok:false` becomes a local error carrying the server's

message, which is how remote method errors reach the caller. Passing `nil` for `out` is valid for methods with no result. Decode failures are wrapped so the caller can tell an output-type problem from a network problem. The deferred close releases the socket on every path.

5. Concurrency and Lifecycle Design

Three standard-library primitives cooperate to make the server safe under concurrent calls and clean shutdown:

Primitive	Where	Why
`sync.Mutex`	`Server.mu`	Protects `listener`, `stopped`, and `acceptDone` from concurrent Start/Stop/Address. It is never held while waiting for a handler, so a slow call cannot block status queries.
`sync.WaitGroup`	`Server.workers`	Counts connection handlers. `Stop` waits until every in-flight call has finished.
Channels	`stopped`, `acceptDone`	`stopped` signals shutdown; `acceptDone` confirms the accept loop exited. Closing channels broadcasts to all waiters without polling.

Shutdown ordering

1. Acquire the mutex, verify the server is running, and snapshot the listener and channels.
2. Clear the lifecycle fields so a repeat `Stop` is a no-op.
3. Close `stopped` to signal the accept loop, then release the mutex.
4. Close the listener. A blocked `Accept` returns an error and the loop exits.
5. Wait for `acceptDone`. At this point no new worker can be registered.
6. Wait for `workers`. Existing calls finish normally and receive their responses.

> **Why the wait order matters.** If `workers.Wait()` ran first, the accept loop could call `Add` while a wait was already in progress. Go forbids adding to a WaitGroup that is being waited on when the counter is zero. Waiting for `acceptDone` first establishes the invariant that all `Add` calls have already happened.

Restart safety. `Start` creates new channels and a new listener on every run. The only state that survives is the receiver object and the `WaitGroup`, whose counter is guaranteed to be zero after `Stop`. This is why `TestStopAndRestart` passes.

Race freedom. `go test -race ./...` reports no data races. The mutex covers every read and write of the lifecycle fields, and the accept loop receives its listener and channels as immutable arguments.

6. Robustness and Security

The server treats every byte from the network as untrusted. The table below maps each failure mode to the mechanism that handles it.

Failure / attack	Mechanism	Result
Malformed JSON	Decode error path in `handle`	JSON error response; connection closed.
Empty or unsafe method name	`validateRequest`	Error response before reflection.

Failure / attack	Mechanism	Result
Too many arguments	`maxArguments` check	Error response; no allocation of parameters.
Oversized argument or request	`maxArgumentBytes`, `maxRequestBytes`, `LimitReader`	Error response; memory use bounded.
Wrong JSON type for a parameter	`json.Unmarshal` error in `invoke`	Error naming the argument index.
Unknown or unexported method	`MethodByName` validity check	`unknown method` error.
Remote method returns an error	Final-error handling in `invoke`	Client receives the error text.
Remote method panics	`callMethodSafely` recover	Error response; server stays alive.
Slow or stuck method	Client `SetDeadline`	Client times out; server shutdown still waits for the handler.
Duplicate Start / repeated Stop	State checks under `mu`	Clear error or idempotent success.
Shutdown during a call	`workers.Wait`	In-flight call completes before Stop returns.

> **Defence in depth.** Validation and the read limit overlap intentionally. The `LimitReader` protects the process regardless of how the request is parsed, while `validateRequest` produces precise, user-facing errors for limit violations. Panic recovery is a third layer: even a bug the validation cannot predict does not take down the server.

7. Testing and Verification

The provided suite in `rpc/rpc_test.go` is the specification. It was not modified. Run it from the module root:

```
cd lab1
go test ./...
go test -race ./...
```

7.1 Test Matrix

Test	What it proves
`TestValidateRequest`	Empty method, whitespace, control characters, too many arguments, one large argument, and an oversized total are all rejected; valid requests pass.
`TestServerClient`	Basic Add and Ping round trips; a remote error from Ping(-1) reaches the client.
`TestServerClientMethodShapes`	Zero arguments, no result, one value, and error-only methods all work.
`TestInvalidRemoteCallsBecomeErrors`	Unknown method, wrong argument count, wrong JSON argument type, and an output type that cannot hold the result each produce an error.
`TestRemotePanicBecomesError`	A panicking method returns an error containing "panicked"; the server survives.

Test	What it proves
`TestConcurrentCalls`	Twenty concurrent calls complete without interference.
`TestStopAndRestart`	Address is empty after Stop; Start works again afterwards.
`TestServerLifecycleErrors`	Address before Start is empty; Stop before Start succeeds; duplicate Start fails; double Stop succeeds.
`TestStopWaitsForActiveHandler`	Stop blocks until a 150 ms handler finishes, and the call still succeeds.
`TestClientTimeout`	A 20 ms client timeout fails against a 150 ms method instead of hanging.
`TestClientConnectionFailures`	An unavailable server produces a connection error.
`TestMalformedRequestGetsErrorResponse`	Raw "not json" receives a JSON error response.
`TestOversizedRequestGetsErrorResponse`	A request with a ~1 MiB argument receives an error response.

7.2 Results

```
PASS
ok      lab1/rpc    0.318s

$ go test -race ./...
?       lab1/cmd/ping    [no test files]
ok      lab1/rpc        1.398s
```

Environment: Go 1.23.4 linux/amd64. The module targets Go 1.20 and uses only the standard library.

7.3 Demo Run

```
$ go run ./cmd/ping
Pong1
Pong2
Pong3
Pong4
```

The demo is an end-to-end check: it starts a real TCP server on an OS-selected port, creates a client from `Server.Address()`, and performs four remote calls.

8. Usage Guide

8.1 Prerequisites

- Go 1.20 or newer available on the PATH (`go version`).
- No external modules: the implementation only imports the standard library.

8.2 Run the Demo

```
cd lab1
go run ./cmd/ping
```

Expected output:

```
Pong1
Pong2
```

```
Pong3
Pong4
```

8.3 Use the Library

Server side. Define a pointer receiver with exported methods. Any `func` shape listed in the assignment is supported automatically.

```
package main

import (
    "log"
    "lab1/rpc"
)

type Calculator struct{}

func (*Calculator) Add(left, right int) int { return left + right }
func (*Calculator) Echo(value string) string { return value }

func main() {
    server, err := rpc.NewServer(&Calculator{})
    if err != nil {
        log.Fatal(err)
    }
    if err := server.Start("127.0.0.1:0"); err != nil {   // port 0 = OS chooses
        log.Fatal(err)
    }
    defer server.Stop()

    log.Println("listening on", server.Address())
    select {}   // keep the process alive
}
```

Client side. Create a client with the server address and call methods by name. Pass a pointer as `out` to receive a result, or `nil` when the method has no result.

```
client, err := rpc.NewClient(server.Address())
if err != nil {
    log.Fatal(err)
}

var sum int
if err := client.Call("Add", &sum, 20, 22); err != nil {
    log.Fatal(err)
}
log.Println("20 + 22 =", sum)          // 42

var echoed string
if err := client.Call("Echo", &echoed, "hello"); err != nil {
    log.Fatal(err)
}
log.Println(echoed)                    // hello

if err := client.Call("NoResult", nil); err != nil {   // nil out
    log.Fatal(err)
}
```

8.4 Protocol-Level Usage

Because the protocol is plain JSON over TCP, any language can call the server. A raw request looks like this:

```
{"method":"Add","args":[2,3]}
{"ok":true,"value":5}
```

```
{"method":"Missing","args":[]}
{"ok":false,"error":"unknown method \"Missing\""}
```

8.5 Running the Tests

```
cd lab1
go test ./...            # functional tests
go test -race ./...      # concurrency safety
go test -run TestStop -v # a focused subset
```

9. Design Decisions and Trade-offs

Decision	Benefit	Trade-off
One request per TCP connection	Trivial framing (one JSON value), no multiplexing state, each call gets its own deadline, and a Client is safe for concurrent use.	A new connection per call adds handshake overhead; unsuitable for very high call rates without a connection pool.
JSON wire format	Human-readable, language-neutral, easy to debug with `nc` or a browser, and directly supported by Go.	Larger and slower than a binary format; numbers lose exactness for very large integers unless encoded as strings.
Reflection-based dispatch	One generic server works for any receiver; no generated code or registration boilerplate.	Lookups and calls are slower than direct calls; errors are discovered at runtime rather than compile time.
Fixed size limits	Bounded memory use and a simple, predictable denial-of-service defence.	Legitimate payloads above 1 MiB per argument or 4 MiB total are rejected.
Recover panics into errors	One faulty method cannot crash the server or drop other clients.	A panic becomes a client-visible error; operators should still log it for diagnosis.
Port 0 in tests and demos	The OS picks a free port, so tests never collide and can run in parallel.	The chosen port is only known at runtime, so callers must use `Address()`.
Clear state before waiting in Stop	`Stop` is idempotent and a restart sees a clean server.	A concurrent `Start` during `Stop` is not supported; callers should treat the pair as sequential.

10. Requirements Checklist

Requirement from the handout	Status	Evidence
TCP client-server application	**Done**	`net.Listen`, `net.DialTimeout`
JSON request/response encoding	**Done**	`encoding/json`, fixed field names
Concurrent client handling	**Done**	goroutine per connection, `TestConcurrentCalls`
Safe start and stop	**Done**	mutex, channels, WaitGroup; lifecycle tests
Discover the OS-selected port	**Done**	`Address()` after port 0
Validate requests before execution	**Done**	`validateRequest`; 6 validation tests
Convert JSON arguments to Go values	**Done**	`reflect.New` + `json.Unmarshal`

Requirement from the handout	Status	Evidence
Invoke methods via reflection	**Done**	`MethodByName` + `Call`
Return remote errors without crashing	**Done**	error return handling + recover
Write and run network tests	**Done**	13/13 tests pass, race detector clean
Keep signatures and JSON names	**Done**	no signature or struct tag changes
Keep TODO comments	**Done**	all seven TODO blocks preserved

Appendix A — API Reference

Symbol	Signature	Purpose
`NewServer`	`func NewServer(receiver any) (*Server, error)`	Validate a non-nil pointer receiver.
`Server.Start`	`func (s *Server) Start(address string) error`	Bind, record state, launch the accept loop.
`Server.Address`	`func (s *Server) Address() string`	Actual listening address, or empty.
`Server.Stop`	`func (s *Server) Stop() error`	Stop accepting and wait for active calls.
`Server.handle`	`func (s *Server) handle(connection net.Conn)`	One request in, one response out.
`validateRequest`	`func validateRequest(req request) error`	Enforce protocol limits.
`Server.invoke`	`func (s *Server) invoke(req request) (any, error)`	Reflection lookup, typed decoding, call, result normalisation.
`callMethodSafely`	`func callMethodSafely(method reflect.Value, arguments []reflect.Value) ([]reflect.Value, error)`	Call a method and recover panics.
`isNilValue`	`func isNilValue(value reflect.Value) bool`	Nil check that is safe for all kinds.
`NewClient`	`func NewClient(address string) (*Client, error)`	Create a client with the default timeout.
`Client.Call`	`func (c *Client) Call(method string, out any, args ...any) error`	Perform one remote call.

Appendix B — Full Test Output

```
?       lab1/cmd/ping   [no test files]
=== RUN   TestValidateRequest
=== RUN   TestValidateRequest/empty_method
=== RUN   TestValidateRequest/space_in_method
=== RUN   TestValidateRequest/control_character
=== RUN   TestValidateRequest/too_many_arguments
=== RUN   TestValidateRequest/large_argument
=== RUN   TestValidateRequest/large_request
--- PASS: TestValidateRequest (0.00s)
    --- PASS: TestValidateRequest/empty_method (0.00s)
    --- PASS: TestValidateRequest/space_in_method (0.00s)
    --- PASS: TestValidateRequest/control_character (0.00s)
    --- PASS: TestValidateRequest/too_many_arguments (0.00s)
    --- PASS: TestValidateRequest/large_argument (0.00s)
    --- PASS: TestValidateRequest/large_request (0.00s)
=== RUN   TestServerClient
--- PASS: TestServerClient (0.00s)
=== RUN   TestServerClientMethodShapes
--- PASS: TestServerClientMethodShapes (0.00s)
=== RUN   TestInvalidRemoteCallsBecomeErrors
=== RUN   TestInvalidRemoteCallsBecomeErrors/unknown_method
=== RUN   TestInvalidRemoteCallsBecomeErrors/wrong_argument_count
=== RUN   TestInvalidRemoteCallsBecomeErrors/invalid_argument_JSON_type
=== RUN   TestInvalidRemoteCallsBecomeErrors/invalid_output_type
--- PASS: TestInvalidRemoteCallsBecomeErrors (0.00s)
    --- PASS: TestInvalidRemoteCallsBecomeErrors/unknown_method (0.00s)
    --- PASS: TestInvalidRemoteCallsBecomeErrors/wrong_argument_count (0.00s)
    --- PASS: TestInvalidRemoteCallsBecomeErrors/invalid_argument_JSON_type (0.00s)
    --- PASS: TestInvalidRemoteCallsBecomeErrors/invalid_output_type (0.00s)
=== RUN   TestRemotePanicBecomesError
--- PASS: TestRemotePanicBecomesError (0.00s)
=== RUN   TestConcurrentCalls
--- PASS: TestConcurrentCalls (0.00s)
=== RUN   TestStopAndRestart
--- PASS: TestStopAndRestart (0.00s)
=== RUN   TestServerLifecycleErrors
--- PASS: TestServerLifecycleErrors (0.00s)
=== RUN   TestStopWaitsForActiveHandler
--- PASS: TestStopWaitsForActiveHandler (0.15s)
=== RUN   TestClientTimeout
--- PASS: TestClientTimeout (0.15s)
=== RUN   TestClientConnectionFailures
--- PASS: TestClientConnectionFailures (0.00s)
=== RUN   TestMalformedRequestGetsErrorResponse
--- PASS: TestMalformedRequestGetsErrorResponse (0.00s)
=== RUN   TestOversizedRequestGetsErrorResponse
--- PASS: TestOversizedRequestGetsErrorResponse (0.01s)
PASS
ok      lab1/rpc        0.318s
```